

Chapter 17. Reactive programming



This chapter covers

- Defining reactive programming and discussing the principles of the Reactive Manifesto
- Reactive programming at the application and system levels
- Showing example code using reactive streams and the Java 9 Flow API
- Introducing RxJava, a widely used reactive library
- Exploring the RxJava operations to transform and combine multiple reactive streams
- Presenting marble diagrams that visually document operations on reactive streams

Before we dig into what reactive programming is and how it works, it's helpful to clarify why this new paradigm is of growing importance. A few years ago, the largest applications had tens of servers and gigabytes of data; response times of several seconds and offline-maintenance times measured in hours were considered to be acceptable. Nowadays, this situation is changing rapidly for at least three reasons:

- **Big Data**— Big Data usually is measured in petabytes and increasing daily.
- **Heterogeneous environments**— Applications are deployed in diverse environments ranging from mobile devices to cloud-based clusters running thousands of multicore processors.
- **Use patterns**— Users expect millisecond response times and 100 percent uptime.

Other formats available



Audiobook



...s demands aren't being met by yesterday's situation has become evident especially as the biggest source of internet traffic, and in the near future when such traffic is overtaken by the Internet of Things (IoT).

Reactive programming addresses these issues by allowing you to process and combine streams of data items coming from different systems and

sources in an asynchronous way. In fact, applications written following this paradigm react to data items as they occur, which allows them to be more responsive in their interactions with users. Moreover, the reactive approach can be applied not only to building a single component or application, but also to coordinating many components into a whole reactive system. Systems engineered in this way can exchange and route messages in varying network conditions and provide availability under heavy load while taking into consideration failures and outages. (Note that although developers traditionally see their systems or applications as being built from components, in this new mashup, loosely coupled style of building systems, these components are often whole applications themselves. Hence, *components* and *applications* are near synonyms.)

The features and advantages that characterize reactive applications and systems are crystallized in the Reactive Manifesto, which we discuss in the next section.

17.1. The Reactive Manifesto

The Reactive Manifesto (<https://www.reactivemanifesto.org>)—developed in 2013 and 2014 by Jonas Bonér, Dave Farley, Roland Kuhn, and Martin Thompson—formalized a set of core principles for developing reactive applications and systems. The Manifesto identified four characteristic features:

- **Responsive**— A reactive system has a fast and, even more important, consistent, predictable response time. As a result, the user knows what to expect. This fact in turn increases user confidence, which is without a doubt the key aspect of a usable application.
- **Resilient**— A system has to remain responsive despite failures. The Reactive Manifesto suggests different techniques to achieve resiliency, including replicating the execution of components, decoupling these components in time (sender and receiver have independent life cycles) and space (sender and receiver run in different processes), and letting each component asynchronously delegate tasks to other components.
- **Elastic**— Another issue that harms the responsiveness of applications is the fact that they can be subject to different workloads during their life cycles. Reactive systems are designed to react automatically to a heavier workload by increasing the number of resources allocated to

Other formats available



Audiobook

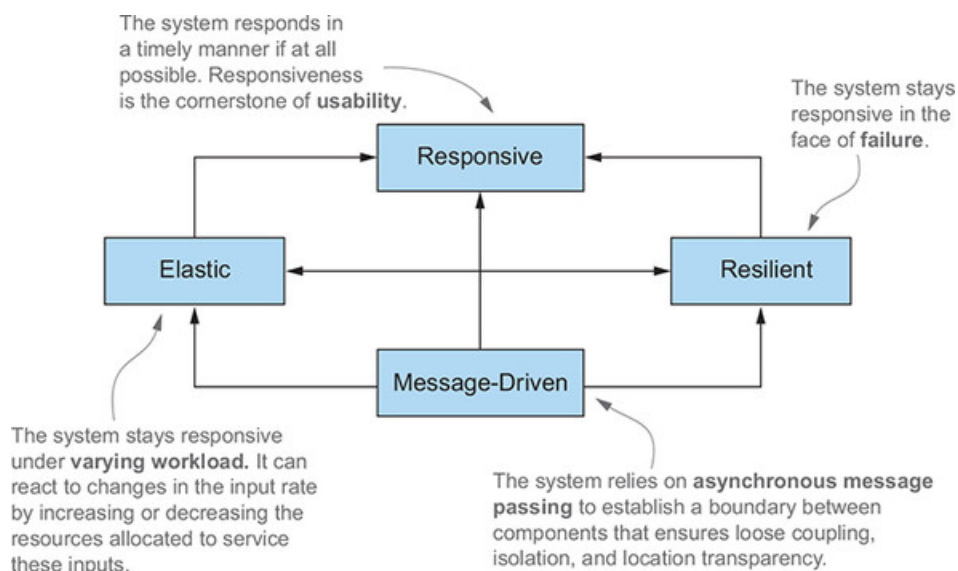


and elasticity require the boundaries of the system to be clearly defined to ensure location transparency. Communication across these boundaries is performed through asynchronous message passing. This choice enables both resiliency (by delegating failures as messages) and elasticity (by monitoring the number of exchanged

messages and then scaling the number of the resources intended to manage them accordingly).

Figure 17.1 shows how these four features are related and dependent on one another. These principles are valid at different scales, from structuring the internals of a small application to determining how these applications have to be coordinated to build a large system. Specific points concerning the level of granularity where these ideas are applied, however, deserve to be discussed in further detail.

Figure 17.1. The key features of a reactive system



17.1.1. Reactive at application level

The main feature of reactive programming for application-level components allows tasks to be executed asynchronously. As we discuss in the rest of this chapter, processing streams of events in an asynchronous and nonblocking way is essential for maximizing the use rate of modern multicore CPUs and, more precisely, of the threads competing for their use. To achieve this goal, the reactive frameworks and libraries share threads (relatively expensive and scarce resources) among lighter constructs such as futures; actors; and (more commonly) event loops dispatching a sequence of callbacks intended to aggregate, transform, and manage the events to be processed.

Background knowledge check

Other formats available



Audiobook



read on.

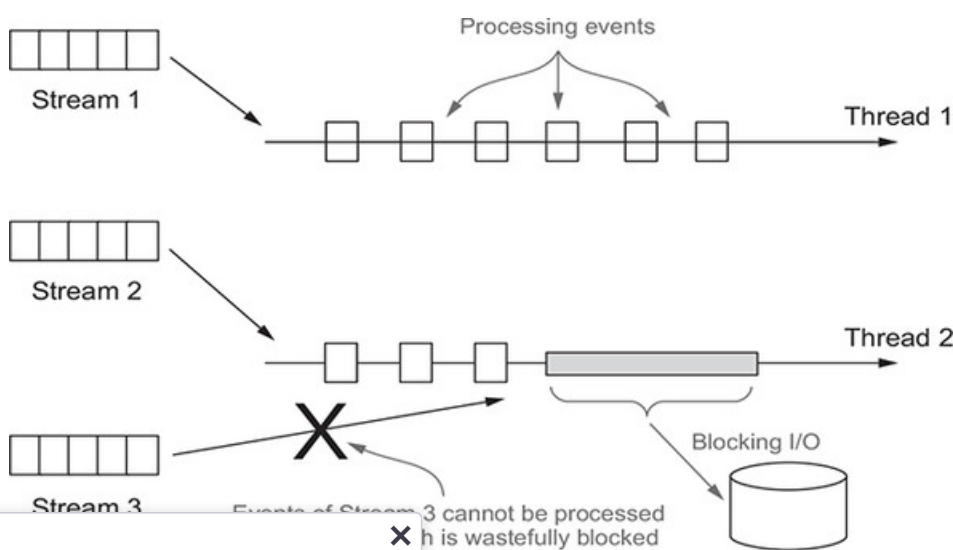
✕ as event, message, signal, and event loop and backpressure, which are used later in a gentler introduction in **chapter 15**. If not,

These techniques not only have the benefit of being cheaper than threads, but also have a major advantage from developers' point of view: they raise the level of abstraction of implementing concurrent and asynchronous applications, allowing developers to concentrate on the business requirements instead of dealing with typical problems of low-level multithreading issues such as synchronization, race conditions, and deadlocks.

The most important thing to pay attention to when using these thread-multiplexing strategies is to never perform blocking operations inside the main event loop. It's helpful to include as blocking operations all I/O-bound operations such as accessing a database or the file system or calling a remote service that may take a long or unpredictable time to complete. It's easy and interesting to explain why you should avoid blocking operations by providing a practical example.

Imagine a simplified yet typical multiplexing scenario with a pool of two threads processing three streams of events. Only two streams can be processed at the same time and the streams have to compete to share those two threads as fairly and efficiently as possible. Now suppose that processing one stream's event triggers a potentially slow I/O operation, such as writing into the file system or retrieving data from a database by using a blocking API. As [figure 17.2](#) shows, in this situation Thread 2 is wastefully blocked waiting for the I/O operation to complete, so although the Thread 1 can process the first stream, the third stream can't be processed before the blocking operation finishes.

Figure 17.2. A blocking operation wastefully keeps a thread busy preventing it from performing other computations.



Other formats available



Audiobook



reactive frameworks (such as RxJava and Akka) allow blocking operations to be executed by means of a separate dedicated thread pool. All the threads in the main pool are free to run uninterrupted, keeping all the cores of the CPU at the highest possible use rate. Keeping separate thread pools for CPU-bound and I/O-bound opera-

tions has the further benefit of allowing you to size and configure the pools with a finer granularity and to monitor the performance of these two kinds of tasks more precisely.

Developing applications by following the reactive principles is only one aspect of reactive programming and often not even the hardest one. Having a set of beautifully designed reactive applications performing efficiently in isolation is at least as important as making them cooperate in a well-coordinated reactive system.

17.1.2. Reactive at system level

A *reactive system* is a software architecture that allows multiple applications to work as a single coherent, resilient platform and also allows these applications to be sufficiently decoupled so when one of them fails, it doesn't bring down the whole system. The main difference between reactive applications and systems is that the former type usually perform computations based on ephemeral streams of data and are called event-driven. The latter type are intended to compose the applications and facilitate communication. Systems with this property are often referred to as being message-driven.

The other important distinction between messages and events is the fact that messages are directed toward a defined single destination, whereas events are facts that will be received by the components that are registered to observe them. In reactive systems, it's also essential for these messages to be asynchronous to keep the sending and the receiving operations decoupled from the sender and receiver, respectively. This decoupling is a requirement for full isolation between components and is fundamental for keeping the system *responsive* under both failures (*resilience*) and heavy load (*elasticity*).

More precisely, resilience is achieved in reactive architectures by isolating failures in the components where they happen to prevent the malfunctions from being propagated to adjacent components and from there in a catastrophic cascade to the rest of the system. Resilience in this reactive sense is more than fault-tolerance. The system doesn't gracefully degrade but fully recovers from failures by isolating them and bringing the system back to a healthy state. This "magic" is obtained by containing the errors and reifying them as messages that are sent to other components acting as supervisors. In this way, the management of the problem can be

Other formats available



external to the failing component itself. A key for resilience, the main enabler for elasticity is *location transparency*, which allows any component of a reactive system to communicate with any other service, regardless of where the recipient resides. Location transparency in turn allows the system to replicate and (automatically) scale any application depending on the cur-

rent workload. Such location-agnostic scaling shows another difference between reactive applications (asynchronous and concurrent and decoupled in time) and reactive systems (which can become decoupled in space through location transparency).

In the rest of this chapter, you put some of these ideas into practice with a few examples of reactive programming, and in particular, you explore Java 9's Flow API.

17.2. Reactive streams and the Flow API

Reactive programming is programming that uses reactive streams.

Reactive streams are a standardized technique (based on the publish-subscribe, or pub-sub, protocol explained in [chapter 15](#)) to process potentially unbounded streams of data asynchronously, in sequence and with mandatory nonblocking backpressure. Backpressure is a flow-control mechanism used in publish-subscribe to prevent a slow consumer of the events in the stream from being overwhelmed by one or more faster producers. When this situation occurs, it's unacceptable for the component under stress to fail catastrophically or to drop events in an uncontrolled fashion. The component needs a way to ask the upstream producers to slow down or to tell them how many events it can accept and process at a given time before receiving more data.

It's worth noting that the requirement for built-in backpressure is justified by the asynchronous nature of the stream processing. In fact, when synchronous invocations are being performed, the system is implicitly backpressured by the blocking APIs. Unfortunately, this situation prevents you from executing any other useful task until the blocking operation is complete, so you end up wasting a lot of resources by waiting. Conversely, with asynchronous APIs you can maximize the use rate of your hardware, but run the risk of overwhelming some other slower downstream component. Backpressure or flow-control mechanisms come into play in this situation; they establish a protocol that prevents data recipients from being overwhelmed without having to block any threads.

These requirements and the behavior that they imply were condensed in the Reactive Streams^[1] project (www.reactive-streams.org), which involved engineers from Netflix, Red Hat, Twitter, Lightbend, and other companies, and produced the definition of four interrelated interfaces

Other formats available



representing the minimal set of features that any Reactive Streams implementation must support. These interfaces are now part of Java 9, nested under `java.util.concurrent.Flow` class, and implemented including Akka Streams (Lightbend), Reactor (Pivotal), RxJava (Netflix), and Vert.x (Red Hat). In the next section, we examine in detail the methods declared by these interfaces and clarify how they're expected to be used to express reactive components.

We capitalize for the Reactive Streams project, but use reactive streams for the concept.

17.2.1. Introducing the Flow class

Java 9 adds one new class for reactive programming: `java.util.concurrent.Flow`. This class contains only static components and can't be instantiated. The `Flow` class contains four nested interfaces to express the publish-subscribe model of reactive programming as standardized by the Reactive Streams project:

- `Publisher`
- `Subscriber`
- `Subscription`
- `Processor`

The `Flow` class allows interrelated interfaces and static methods to establish flow-controlled components, in which `Publishers` produce items consumed by one or more `Subscribers`, each managed by a `Subscription`. The `Publisher` is a provider of a potentially unbounded number of sequenced events, but it's constrained by the backpressure mechanism to produce them according to the demand received from its `Subscriber(s)`. The `Publisher` is a Java functional interface (declares only one single abstract method) that allows a `Subscriber` to register itself as a listener of the events issued by the `Publisher`; flow control, including backpressure, between `Publishers` and `Subscribers` is managed by a `Subscription`. These three interfaces, along with the `Processor` interface, are captured in [listings 17.1, 17.2, 17.3](#), and [17.4](#).

Listing 17.1. The `Flow.Publisher` interface

```
@FunctionalInterface
public interface Publisher<T> {
    void subscribe(Subscriber<? super T> s);
}
```

On the other side, the `Subscriber` interface has four callback methods that are invoked by the `Publisher` when it produces the corresponding

Other formats available



Audiobook



Subscriber interface

```
public interface Subscriber<T> {
    void onSubscribe(Subscription s);
    void onNext(T t);
    void onError(Throwable t);
}
```

```

        void onComplete();
    }

```

Those events have to be published (and the corresponding methods invoked) strictly following the sequence defined by this protocol:

```

onSubscribe onNext* (onError | onComplete)?

```

This notation means that `onSubscribe` is always invoked as the first event, followed by an arbitrary number of `onNext` signals. The stream of events can go on forever, or it can be terminated by an `onComplete` callback to signify that no more elements will be produced or by an `onError` if the `Publisher` experiences a failure. (Compare reading from a terminal when you get a string or an indication of an end-of-file or I/O error.)

When a `Subscriber` registers itself on a `Publisher`, the `Publisher`'s first action is to invoke the `onSubscribe` method to pass back a `Subscription` object. The `Subscription` interface declares two methods. The `Subscriber` can use the first method to notify the `Publisher` that it's ready to process a given number of events; the second method allows it to cancel the `Subscription`, thereby telling the `Publisher` that it's no longer interested in receiving its events.

Listing 17.3. The `Flow.Subscription` interface

```

public interface Subscription {
    void request(long n);
    void cancel();
}

```

The Java 9 `Flow` specification defines a set of rules through which the implementations of these interfaces should cooperate. These rules can be summarized as follows:

- The `Publisher` must send the `Subscriber` a number of elements no greater than that specified by the `Subscription`'s `request` method. A `Publisher`, however, may send fewer `onNext` than requested and terminate the `Subscription` by calling `onComplete` if the operation terminated successfully or `onError` if it failed. In these cases, when a terminator signal (either `onComplete` or `onError`) is received, the `Publisher` must not send any further signals to its `Subscribers`, and the `Subscription` is considered to be canceled.
- The `Subscriber` must notify the `Publisher` that it's ready to receive and process `n` elements. In this way, the `Subscriber` exercises back-pressure on the `Publisher` preventing the `Subscriber` from being

Other formats available

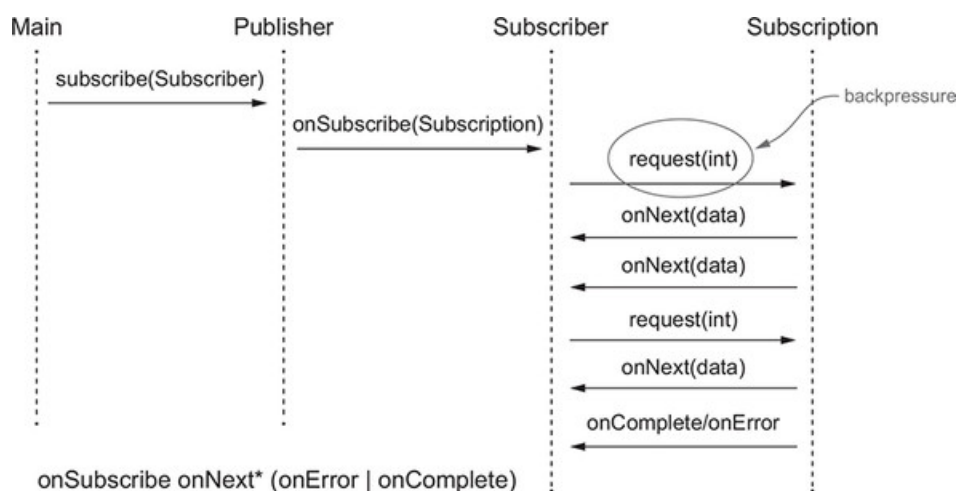
 Audiobook 

overwhelmed by too many events to manage. Moreover, when processing the `onComplete` or `onError` signals, the `Subscriber` isn't allowed to call any method on the `Publisher` or `Subscription` and must consider the `Subscription` to be canceled. Finally, the `Subscriber` must be prepared to receive these terminal signals even without any preceding call of the `Subscription.request()` method and to receive one or more `onNext` even after having called `Subscription.cancel()`.

- The `Subscription` is shared by exactly one `Publisher` and `Subscriber` and represents the unique relationship between them. For this reason, it must allow the `Subscriber` to call its `request` method synchronously from both the `onSubscribe` and `onNext` methods. The standard specifies that the implementation of the `Subscription.cancel()` method has to be idempotent (calling it repeatedly has the same effect as calling it once) and thread-safe so that, after the first time it has been called, any other additional invocation on the `Subscription` has no effect. Invoking this method asks the `Publisher` to eventually drop any references to the corresponding `Subscriber`. Resubscribing with the same `Subscriber` object is discouraged, but the specification doesn't mandate an exception being raised in this situation because all previously canceled subscriptions would have to be stored indefinitely.

Figure 17.3 shows the typical life cycle of an application implementing the interfaces defined by the Flow API.

Figure 17.3. The life cycle of a reactive application using the Flow API



The fourth and final member of the `Flow` class is the `Processor` interface, which allows a `Publisher` and `Subscriber` without requiring

Other formats available



Audiobook



Listing 17.4. The `Flow.Processor` interface

```
public interface Processor<T, R> extends Subscriber<T>, Publisher<R> { }
```

In fact, this interface represents a transformation stage of the events processed through the reactive stream. When receiving an error, the Processor can choose to recover from it (and then consider the Subscription to be canceled) or immediately propagate the `onError` signal to its Subscriber(s). The Processor should also cancel its upstream Subscription when its last Subscriber cancels its Subscription to propagate the cancellation signal (even though this cancellation isn't strictly required by the specification).

The Java 9 Flow API/Reactive Streams API mandates that any implementation of all the methods of the Subscriber interface should never block the Publisher, but it doesn't specify whether these methods should process the events synchronously or asynchronously. Note, however, that all methods defined by these interfaces return `void` so that they can be implemented in a completely asynchronous way.

In this next section, you try to put to work what you've learned so far through a simple, practical example.

17.2.2. Creating your first reactive application

The interfaces defined in the Flow class are, in most cases, not intended to be implemented directly. Unusually, the Java 9 library doesn't provide classes that implement them either! These interfaces are implemented by the reactive libraries that we've already mentioned (Akka, RxJava, and so on). The Java 9 specification of `java.util.concurrent.Flow` works both as a contract to which all those libraries must adhere and a lingua franca allowing reactive applications developed on top of different reactive libraries to cooperate and talk to one another. Moreover, those reactive libraries typically offer many more features (classes and methods that transform and merge reactive streams beyond the minimal subset specified by the `java.util.concurrent.Flow` interface).

That being said, it makes sense for you to develop a first reactive application directly on top of the Java 9 Flow API to get a feeling for how the four interfaces discussed in the preceding sections work together. To this end, you'll write a simple temperature-reporting program using reactive principles. This program has two components:

- TempInfo, which mimics a remote thermometer (constantly reporting values between 0 and 99 degrees Fahrenheit, cities most of the time) responds to these reports and prints the stream of temperatures reported by a sensor installed in a given city

Other formats available



Audiobook



The first step is defining a simple class that conveys the currently reported temperature, as shown in the following listing.

Listing 17.5. A Java bean conveying the currently reported temperature

```
import java.util.Random;

public class TempInfo {

    public static final Random random = new Random();

    private final String town;
    private final int temp;

    public TempInfo(String town, int temp) {
        this.town = town;
        this.temp = temp;
    }

    public static TempInfo fetch(String town) {
        if (random.nextInt(10) == 0)
            throw new RuntimeException("Error!");
        return new TempInfo(town, random.nextInt(100));
    }

    @Override
    public String toString() {
        return town + " : " + temp;
    }

    public int getTemp() {
        return temp;
    }

    public String getTown() {
        return town;
    }
}
```

- **1** TempInfo instance for a given town is created via a static factory method.
- **2** Fetching the current temperature may randomly fail one time out of ten.
- **3** Returns a random temperature in the range 0 to 99 degrees Fahrenheit

Other formats available



Audiobook



✕ In model, you can implement a
ures of a given town that sends a temper-
rt is requested by its Subscriber as

shown in the following listing.

Listing 17.6. A Subscription sending a stream of TempInfo to its Subscriber

```

import java.util.concurrent.Flow.*;

public class TempSubscription implements Subscription {

    private final Subscriber<? super TempInfo> subscriber;
    private final String town;

    public TempSubscription( Subscriber<? super TempInfo> subscriber,
                            String town ) {
        this.subscriber = subscriber;
        this.town = town;
    }

    @Override
    public void request( long n ) {
        for (long i = 0L; i < n; i++) {
            try {
                subscriber.onNext( TempInfo.fetch( town ) );
            } catch (Exception e) {
                subscriber.onError( e );
                break;
            }
        }
    }

    @Override
    public void cancel() {
        subscriber.onComplete();
    }
}

```

- **1 Loops once per request made by the Subscriber**
- **2 Sends the current temperature to the Subscriber**
- **3 In case of a failure while fetching the temperature propagates the error to the Subscriber**
- **4 If the subscription is canceled, send a completion (onComplete) signal to the Subscriber.**

The next step is creating a Subscriber that, every time it gets a new element, prints the temperatures received from the Subscription and asks for a new report as shown in the next listing.

Listing 17.7. A Subscriber printing the received temperatures

Other formats available



```

rent.Flow.*;

```

```

iber implements Subscriber<TempInfo> {

```

```

private Subscription subscription;

```

```

@Override

```

```

        public void onSubscribe( Subscription subscription ) { 1
            this.subscription = subscription;
            subscription.request( 1 );
        }

        @Override
        public void onNext( TempInfo tempInfo ) { 2
            System.out.println( tempInfo );
            subscription.request( 1 );
        }

        @Override
        public void onError( Throwable t ) { 3
            System.err.println(t.getMessage());
        }

        @Override
        public void onComplete() {
            System.out.println("Done!");
        }
    }
}

```

- **1 Stores the subscription and sends a first request**
- **2 Prints the received temperature and requests a further one**
- **3 Prints the error message in case of an error**

The next listing puts your reactive application to work with a Main class that creates a Publisher and then subscribes to it by using TempSubscriber.

Listing 17.8. A main class: creating a Publisher and subscribing TempSubscriber to it

```

import java.util.concurrent.Flow.*;

public class Main {
    public static void main( String[] args ) {
        getTemperatures( "New York" ).subscribe( new TempSubscriber() );
    }

    private static Publisher<TempInfo> getTemperatures( String town ) {
        return subscriber -> subscriber.onSubscribe(
            new TempSubscription( subscriber, town )
        );
    }
}

```

Other formats available



Audiobook

f temperatures in New York and sub-

scribes the TempSubscriber to it

- **2 Returns a Publisher that sends a TempSubscription to the Subscriber that subscribes to it**

Here, the `getTemperatures` method returns a lambda expression that takes a `Subscriber` as an argument and invokes its `onSubscribe` method, passing to it a new `TempSubscription` instance. Because the signature of this lambda is identical to the only abstract method of the `Publisher` functional interface, the Java compiler can automatically convert the lambda to a `Publisher` (as you learned in [chapter 3](#)). The `main` method creates a `Publisher` for the temperatures in New York and then subscribes a new instance of the `TempSubscriber` class to it. Running `main` produces output something like this:

```
New York : 44
New York : 68
New York : 95
New York : 30
Error!
```

In the preceding run, `TempSubscription` successfully fetched the temperature in New York four times but failed on the fifth reading. It seems that you correctly implemented the problem by using three of the four interfaces of the Flow API. But are you sure that there aren't any mistakes in the code? Give this question some thought by completing the following quiz.

Quiz 17.1:

The example developed so far has a subtle problem. This problem, however, is hidden by the fact that at some point, the stream of temperatures will be interrupted by the error randomly generated inside the `TempInfo` factory method. Can you guess what will happen if you comment out the statement generating the random error and let your `main` run long enough?

Answer:

The problem with what you've done so far is that every time the `TempSubscriber` receives a new element into its `onNext` method, it sends a new request to the `TempSubscription`, and then the `request` method sends another element to the `TempSubscriber` itself. These recursive invocations are pushed onto the stack one after the other until the

Other formats available



Audiobook

✕ StackOverflowError like the following:

```
in" java.lang.StackOverflowError
at java.base/java.io.PrintStream.print(PrintStream.java:666)
at java.base/java.io.PrintStream.println(PrintStream.java:820)
at flow.TempSubscriber.onNext(TempSubscriber.java:36)
at flow.TempSubscriber.onNext(TempSubscriber.java:24)
at flow.TempSubscription.request(TempSubscription.java:60)
```

```

    at flow.TempSubscriber.onNext(TempSubscriber.java:37)
    at flow.TempSubscriber.onNext(TempSubscriber.java:24)
    at flow.TempSubscription.request(TempSubscription.java:60)
    ...

```

What can you do to fix this problem and avoid overflowing the stack?

One possible solution is to add an `Executor` to the `TempSubscription` and then use it to send new elements to the `TempSubscriber` from a different thread. To achieve this goal, you can modify the `TempSubscription` as shown in the next listing. (The class is incomplete; the full definition uses the remaining definitions from [listing 17.6](#).)

Listing 17.9. Adding an Executor to the TempSubscription

```

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class TempSubscription implements Subscription {
    1

    private static final ExecutorService executor =
        Executors.newSingleThreadExecutor()

    @Override
    public void request( long n ) {
        executor.submit( () -> {
            2
            for (long i = 0L; i < n; i++) {
                try {
                    subscriber.onNext( TempInfo.fetch( town ) );
                } catch (Exception e) {
                    subscriber.onError( e );
                    break;
                }
            }
        });
    }
}

```

- 1 Unmodified code of original `TempSubscription` has been omitted.
- 2 Sends the next elements to the subscriber from a different

Other formats available



the four interfaces defined by the `Flow` interface? A good example of how to use that interface is to create a `Publisher` that reports the temperatures in Celsius instead of Fahrenheit (for subscribers outside the United States).

17.2.3. Transforming data with a Processor

As described in [section 17.2.1](#), a Processor is both a Subscriber and a Publisher. In fact, it's intended to subscribe to a Publisher and republish the data that it receives after transforming that data. As a practical example, implement a Processor that subscribes to a Publisher that emits temperatures in Fahrenheit and republishes them after converting them to Celsius, as shown in this next listing.

Listing 17.10. A Processor transforming temperatures from Fahrenheit to Celsius

```
import java.util.concurrent.Flow.*;

public class TempProcessor implements Processor<TempInfo, TempInfo> {

    private Subscriber<? super TempInfo> subscriber;

    @Override
    public void subscribe( Subscriber<? super TempInfo> subscriber ) {
        this.subscriber = subscriber;
    }

    @Override
    public void onNext( TempInfo temp ) {
        subscriber.onNext( new TempInfo( temp.getTown(),
                                           (temp.getTemp() - 32) * 5 / 9) );
    }

    @Override
    public void onSubscribe( Subscription subscription ) {
        subscriber.onSubscribe( subscription );
    }

    @Override
    public void onError( Throwable throwable ) {
        subscriber.onError( throwable );
    }

    @Override
    public void onComplete() {
        subscriber.onComplete();
    }
}
```

- 1 A processor transforming a TempInfo into another TempInfo after converting the temperature to

Other formats available



Audiobook



ated unchanged to the upstream

subscriber.

Note that the only method of the TempProcessor that contains some business logic is onNext, which republishes temperatures after convert-

ing them from Fahrenheit to Celsius. All other methods that implement the Subscriber interface merely pass on unchanged (delegate) all received signals to the upstream Subscriber, and the Publisher's subscribe method registers the upstream Subscriber into the Processor.

The next listing puts the TempProcessor to work by using it in your Main class.

Listing 17.11. Main class: create a Publisher and subscribe TempSubscriber to it

```
import java.util.concurrent.Flow.*;

public class Main {
    public static void main( String[] args ) {
        getCelsiusTemperatures( "New York" )
            .subscribe( new TempSubscriber() );
    }

    public static Publisher<TempInfo> getCelsiusTemperatures(String town) {
        return subscriber -> {
            TempProcessor processor = new TempProcessor();
            processor.subscribe( subscriber );
            processor.onSubscribe( new TempSubscription(processor, town)
        };
    }
}
```

- **1 Creates a new Publisher of Celsius temperatures for New York**
- **2 Subscribes the TempSubscriber to the Publisher**
- **3 Creates a TempProcessor and puts it between the Subscriber and returned Publisher**

This time, running Main produces the following output, with temperatures that are typical of the Celsius scale:

```
New York : 10
New York : -12
New York : 23
Error!
```

Other formats available



Audiobook



plemented the interfaces defined in the
became familiar with asynchronous
h-subscribe protocol that forms the core
was something slightly unusual about this

example, which we turn to in the next section.

17.2.4. Why doesn't Java provide an implementation of the Flow API?

The Flow API in Java 9 is rather odd. The Java library generally provides interfaces and implementations for them, but here, you’ve implemented the Flow API yourself. Let’s make a comparison with the List API. As you know, Java provides the `List<T>` interface that’s implemented by many classes, including `ArrayList<T>`. More precisely (and rather invisibly to the user) the class `ArrayList<T>` extends the abstract class `AbstractList<T>`, which implements the interface `List<T>`. By contrast, Java 9 declares the interface `Publisher<T>` and provides no implementation, which is why you had to define your own (apart from the learning benefit you got from implementing it). Let’s face it—an interface on its own may help you structure your programming thoughts, but it doesn’t help you write programs any faster!

What’s going on? The answer is historic: there were multiple Java code libraries of reactive streams (such as Akka and RxJava). Originally, these libraries were developed separately, and although they implemented reactive programming via publish-subscribe ideas, they used different nomenclature and APIs. During the standardization process of Java 9, these libraries evolved so that their classes formally implemented the interfaces in `java.util.concurrent.Flow`, as opposed to merely implementing the reactive concepts. This standard enables more collaboration among different libraries.

Note that building a reactive-streams implementation is complex, so most users will merely use an existing one. Like many classes that implement an interface, they typically provide richer functionality than is required for a minimal implementation.

In the next section, you use one of the most widely used libraries: the RxJava (reactive extensions to Java) library developed by Netflix, specifically the current RxJava 2.0 version, which implements the Java 9 Flow interfaces.

17.3. Using the reactive library RxJava

RxJava was among one the first libraries to develop reactive applications in Java. It was born at Netflix as a port of the Reactive Extensions (Rx) project, originally developed by Microsoft in the .Net environment. RxJava version 2.0 was adjusted to adhere to the Reactive Streams API explained earlier in this chapter and adopted by Java 9 as `java.util.con-`

Other formats available



Audiobook



...y in Java, this fact is apparent from the
...w interfaces, for example, including
Publisher with a line such as this:

```
import java.lang.concurrent.Flow.*;
```

But you also need to import the appropriate implementing classes with a line such as

```
import io.reactivex.Observable;
```

if you want to use the `Observable` implementation of `Publisher`, as you'll choose to do later in this chapter.

We must emphasize one architectural issue: good systems-architectural style avoids making visible throughout the system any fine-details concepts that are used in only one part of the system. Accordingly, it's good practice to use an `Observable` only where the additional structure of an `Observable` is required and otherwise use its interface of `Publisher`. Note that you observe this guideline with the `List` interface without thinking. Even though a method may have been passed a value that you know to be an `ArrayList`, you declare the parameter for this value to be of type `List`, so you avoid exposing and constraining the implementation details. Indeed, you allow a later change of implementation from `ArrayList` to `LinkedList` not to require ubiquitous changes.

In the rest of this section, you define a temperature-reporting system by using RxJava's implementation of reactive streams. The first issue you come across is that RxJava provides two classes, both of which implement `Flow.Publisher`.

On reading the RxJava documentation, you find that one class is the `io.reactivex.Flowable` class, which includes the reactive pull-based backpressure feature of Java 9 `Flow` (using `request`) exemplified in [list-ings 17.7](#) and [17.9](#). Backpressure prevents a `Subscriber` from being overrun by data being produced by a fast `Publisher`. The other class is the original RxJava `io.reactivex.Observable` version of `Publisher`, which didn't support backpressure. This class is both simpler to program and more appropriate for user-interface events (such as mouse movements); these events are streams that can't be reasonably backpressured. (You can't ask the user to slow down or stop moving the mouse!) For this reason, RxJava provides these two implementing classes for the common idea stream of events.

The RxJava advice is to use the nonbackpressured `Observable` when you

Other formats available



✕ thousand elements or when you're are mouse moves or touch events, which are aren't frequent anyway.

Because we analyzed the backpressure scenario while discussing the `Flow` API in the preceding section, we won't discuss `Flowable` anymore; instead, we'll demonstrate the `Observable` interface at work in a use

case without backpressure. It's worth noting that any subscriber can effectively turn off backpressuring by invoking `request(Long.MAX_VALUE)` on the subscription, even if this practice isn't advisable unless you're sure that the `Subscriber` will always be able to process all the received events in a timely manner.

17.3.1. Creating and using an Observable

The `Observable` and `Flowable` classes come with many convenient factory methods that allow you to create many types of reactive streams. (Both `Observable` and `Flowable` implement `Publisher`, so these factory methods publish reactive streams.)

The simplest `Observable` that you may want to create is made of a fixed number of predetermined elements, as follows:

```
Observable<String> strings = Observable.just( "first", "second" );
```

Here, the `just()` factory method^[2] converts one or more elements to an `Observable` that emits those elements. A subscriber to this `Observable` receives `onNext("first")`, `onNext("second")`, and `onComplete()` messages, in that order.

²

This naming convention is slightly unfortunate, because Java 8 started using `of()` for similar factory methods as popularized by the `Stream` and `Optional` APIs.

Another `Observable` factory method that's quite common, especially when your application interacts with a user in real time, emits events at a fixed time rate:

```
Observable<Long> onePerSec = Observable.interval(1, TimeUnit.SECONDS);
```

The `interval` factory method returns an `Observable`, named `onePerSec`, that emits an infinite sequence of ascending values of type `long`, starting at zero, at a fixed time interval of your choosing (1 second in this example). Now plan to use `onePerSec` as the basis of another

Other formats available

 Audiobook 

temperature reported for a given town each

to achieve this goal, you can print those temperatures each second. To do so, you need to subscribe to `onePerSec` to be notified by it every time a second has passed and then fetch and print the temperatures of the town of interest. In RxJava, the `Observable`^[3] plays

the role of the Publisher in the Flow API, so the Observer similarly corresponds to Flow's Subscriber interface. The RxJava Observer interface declares the same methods as the Java 9 Subscriber given in [listing 17.2](#), with the difference that the onSubscribe method has a Disposable argument rather than a Subscription. As we mentioned earlier, Observable doesn't support backpressure, so it doesn't have a request method that forms part of a Subscription. The full Observer interface is

3

Note that the Observer interface and the Observable class have been deprecated since Java 9. New code should use the Flow API. It remains to be seen how RxJava will evolve.

```
public interface Observer<T> {
    void onSubscribe(Disposable d);
    void onNext(T t);
    void onError(Throwable t);
    void onComplete();
}
```

Note, however, that RxJava's API are more flexible (have more over-loaded variants) than the native Java 9 Flow API. You can subscribe to an Observable, for example, by passing a lambda expression with the signature of the onNext method and omitting the other three methods. In other words, you can subscribe to an Observable with an Observer that implements only the onNext method with a Consumer of the received event, leaving the other methods defaulting to a no-op for completion and error handling. By using this feature, you can subscribe to the Observable onePerSec and use it to print the temperatures in New York once a second, all in a single line of code:

```
onePerSec.subscribe(i -> System.out.println(TempInfo.fetch( "New York"
```

In this statement, the onePerSec Observable emits one event per second. and on receipt of this message, the Subscriber fetches the temperature in New York and prints it. If you put this statement in a main method and try to execute it, however, you see nothing because the Observable

Other formats available



✕ is executed in a thread that belongs to pool, which is made up of daemon threads. terminates immediately and, in doing so, kills

the daemon thread before it can produce any output.

4

This fact doesn't seem to be clear from the documentation, although you can find statements to this effect in the stackoverflow.com on-line developer community.

As a bit of a hack, you can prevent this immediate termination by putting a thread sleep after the preceding statement. Better, you could use the `blockingSubscribe` method that calls the callbacks on the current thread (in this case, the main thread). For the purposes of a running demonstration, `blockingSubscribe` is perfectly suitable. In a production context, however, you normally use the `subscribe` method, as follows:

```
onePerSec.blockingSubscribe(  
    i -> System.out.println(TempInfo.fetch( "New York" ))  
);
```

You may obtain output such as the following:

```
New York : 87  
New York : 18  
New York : 75  
java.lang.RuntimeException: Error!  
at flow.common.TempInfo.fetch(TempInfo.java:18)  
at flow.Main.lambda$main$0(Main.java:12)  
at io.reactivex.internal.observers.LambdaObserver  
    .onNext(LambdaObserver.java:59)  
at io.reactivex.internal.operators.observable  
    .ObservableInterval$IntervalObserver.run(ObservableInterval.j
```

Unfortunately, the temperature fetching may, by design, fail randomly (and indeed does after three readings). Because your `Observer` implements only the happy path and doesn't have any sort of error management, such as `onError`, this failure blows up in the user's face as an uncaught exception.

It's time to raise the bar and start complicating this example a bit. You don't want to add only error management. You also have to generalize what you have. You don't want to print the temperatures immediately but provide users a factory method that returns an `Observable` emitting those temperatures once a second for (say) at most five times before com-

plaining easily by using a factory method named `observableFromLambda`, taking as an argument a lambda returning `void`, as shown in the following

Other formats available



Listing 17.12. Creating an `Observable` emitting temperature once a second

```

public static Observable<TempInfo> getTemperature(String town) {
    return Observable.create(emitter ->
        Observable.interval(1, TimeUnit.SECONDS)
            .subscribe(i -> {
                if (!emitter.isDisposed()) {
                    if ( i >= 5 ) {
                        emitter.onComplete();
                    } else {
                        try {
                            emitter.onNext(TempInfo.fetch(town));
                        } catch (Exception e) {
                            emitter.onError(e);
                        }
                    }
                }
            }
        ));
}

```

- **1 Creates an Observable from a function consuming an Observer**
- **2 An Observable emitting an infinite sequence of ascending longs, one per second**
- **3 Do something only if the consumed observer hasn't been disposed yet (for a former error).**
- **4 If the temperature has been already emitted five times, completes the observer terminating the stream**
- **5 Otherwise, sends a temperature report to the Observer**
- **6 In case of error, notifies the Observer**

Here, you're creating the returned Observable from a function that consumes an ObservableEmitter, sending the desired events to it. The RxJava ObservableEmitter interface extends the basic RxJava Emitter, which you can think of as being an Observer without the onSubscribe method,

```

public interface Emitter<T> {
    void onNext(T t);
    void onError(Throwable t);
    void onComplete();
}

```

with a few more methods to set a new Disposable on the Emitter and check whether the sequence has been already disposed downstream.

Other formats available



observable such as onePerSec that publishes ascending longs, one per second. Inside the subscribing function (passed as an argument to the subscribe method), you first check whether the consumed Observer has been already disposed by the isDisposed method provided by the ObservableEmitter interface. (This situation could happen if an error occurred in an earlier

iteration.) If the temperature has been already emitted five times, the code completes the `Observer`, terminating the stream; otherwise, it sends the most recent temperature report for the requested town to the `Observer` in a try/catch block. If an error occurs during the temperature fetching, it propagates the error to the `Observer`.

Now it's easy to implement a complete `Observer` that will later be used to subscribe to the `Observable` returned by the `getTemperature` method and that prints the temperatures it publishes as shown in the next listing.

Listing 17.13. An `Observer` printing the received temperatures

```
import io.reactivex.Observer;
import io.reactivex.disposables.Disposable;

public class TempObserver implements Observer<TempInfo> {
    @Override
    public void onComplete() {
        System.out.println( "Done!" );
    }

    @Override
    public void onError( Throwable throwable ) {
        System.out.println( "Got problem: " + throwable.getMessage() );
    }

    @Override
    public void onSubscribe( Disposable disposable ) {
    }

    @Override
    public void onNext( TempInfo tempInfo ) {
        System.out.println( tempInfo );
    }
}
```

This `Observer` is similar to the `TempSubscriber` class from [listing 17.7](#) (which implements Java 9's `Flow.Subscriber`), but you have a further simplification. Because RxJava's `Observable` doesn't support backpressure, you don't need to request() further elements after processing the published ones.

Other formats available



main program in which you subscribe this
returned by the `getTemperature` method

Listing 17.14. A `main` class printing the temperatures in New York

```
public class Main {
```

```

        public static void main(String[] args) {
            Observable<TempInfo> observable = getTemperature( "New York" );
            observable.blockingSubscribe( new TempObserver() );
        }
    }
}

```

- **1 Creates an Observable emitting the temperatures reported in New York once a second**
- **2 Subscribes to that Observable with a simple Observer that prints the temperatures**

Supposing that this time, no error occurs while the temperatures are being fetched, `main` prints a line per second five times, and then the `Observable` emits the `onComplete` signal, so you might obtain output like the following:

```

New York : 69
New York : 26
New York : 85
New York : 94
New York : 29
Done!

```

It's time to enrich your RxJava example a bit further and in particular to see how this library allows you to manipulate one or more reactive streams.

17.3.2. Transforming and combining Observables

One of the main advantages of RxJava and other reactive libraries in working with reactive streams, compared with what's offered by the native Java 9 `Flow` API, is that they provide a rich toolbox of functions to combine, create, and filter any of those streams. As we demonstrated in the preceding sections, a stream can be used as an input to another one. Also, you've learned about the Java 9 `Flow.Processor` used in [section 17.2.3](#) to transform temperatures in Fahrenheit to Celsius. But you can also filter a stream to get another one that has only the elements you're interested in, transform those elements with a given mapping function

Other formats available



Audiobook



ed with `Flow.Processor`), or even
in many ways (which can't be achieved

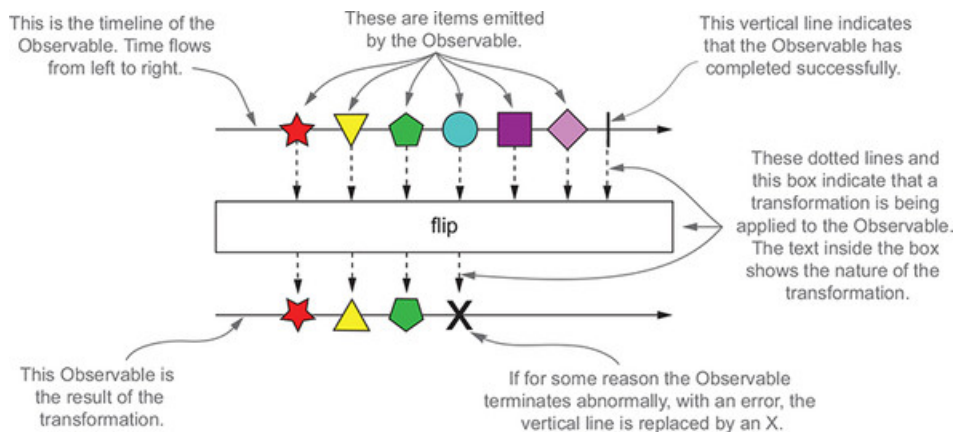
These transforming and combining functions can be quite sophisticated, to the point that explaining their behavior in plain words may result in

awkward, convoluted sentences. To get an idea, see how RxJava documents its `mergeDelayError` function:

Flattens an Observable that emits Observables into one Observable, in a way that allows an Observer to receive all successfully emitted items from all of the source Observables without being interrupted by an error notification from one of them, while limiting the number of concurrent subscriptions to these Observables.

You must admit that what this function does isn't immediately evident. To alleviate this problem, the reactive-streams community decided to document the behaviors of these functions in a visual way, using so-called marble diagrams. A *marble diagram*, such as that shown in [figure 17.4](#), represents the temporally ordered sequence of elements in a reactive stream as geometric shapes on a horizontal line; special symbols represent error and completion signals. Boxes indicate how named operators transform those elements or combine multiple streams.

Figure 17.4. Legend of a marble diagram documenting an operator provided by a typical reactive library



Using this notation, it's easy to provide a visual representation of the features of all the RxJava library's functions as shown in [figure 17.5](#), which exemplifies `map` (which transforms the elements published by an Observable) and `merge` (which combines the events emitted by two or more Observables into one).

Figure 17.5. The marble diagrams for the `map` and `merge` functions



You may wonder how you can use `map` and `merge` to improve and add features to the RxJava example that you developed in the preceding sec-

tion. Using `map` is a more concise way to achieve the transformation from Fahrenheit to Celsius that you implemented by using the Flow API's Processor, as shown in the following listing.

Listing 17.15. Using `map` on `Observable` to transform Fahrenheit into Celsius

```
public static Observable<TempInfo> getCelsiusTemperature(String town) {  
    return getTemperature( town )  
        .map( temp -> new TempInfo( temp.getTown(),  
                                     (temp.getTemp() - 32) * 5 /  
                                     9 ) )  
}
```

This simple method takes the `Observable` returned by the `getTemperature` method of [listing 17.12](#) and returns another `Observable` that re-emits the temperatures published (once a second) by the first one after transforming them from Fahrenheit to Celsius.

To reinforce your understanding of how you can manipulate the elements emitted by an `Observable`, try to use another transforming function in the following quiz.

Quiz 17.2: Filtering only negative temperatures

The `filter` method of the `Observable` class takes a `Predicate` as an argument and produces a second `Observable` that emits only the elements that pass the test defined by that `Predicate`. Suppose that you've been asked to develop a warning system that alerts the user when there's risk of ice. How can you use this operator to create an `Observable` that emits the temperature in Celsius registered in a town only in case the temperature is below zero? (The Celsius scale conveniently uses zero for the freezing point of water.)

Answer:

It's enough to take the `Observable` returned by the method in [listing 17.15](#) and apply to it the `filter` operator with a `Predicate` that accepts only negative temperature as follows:

Other formats available



```
public static Observable<TempInfo> getNegativeTemperature(String town) {  
    return getTemperature( town )  
        .filter( temp -> temp.getTemp() < 0 );  
}
```

Now also imagine that you've been asked to generalize this last method and allow your users to have an `Observable` that emits the temperatures not only for a single town, but also for a set of towns. [Listing 17.16](#) satisfies the last requirement by invoking the method in [listing 17.15](#) once for each town and combining all the `Observables` obtained from these invocations into a single one through the `merge` function.

Listing 17.16. Merging the temperatures reported for one or more towns

```
public static Observable<TempInfo> getCelsiusTemperatures(String... towns) {
    return Observable.merge(Arrays.stream(towns)
        .map(TempObservable::getCelsiusTemperature)
        .collect(toList()));
}
```

This method takes a `varargs` argument containing the set of towns for which you want temperatures. This `varargs` is converted to a stream of `String`; then each `String` is passed to the `getCelsiusTemperature` method of [listing 17.11](#) (improved in [listing 17.15](#)). This way, each town is transformed into an `Observable` emitting the temperature of that town each second. Finally, the stream of `Observables` is collected into a list, and the list is passed to the `merge` static factory method provided by the `Observable` class itself. This method takes an `Iterable` of `Observables` and combines their output so that they act like a single `Observable`. In other words, the resulting `Observable` emits all the events published by all the `Observables` contained in the passed `Iterable`, preserving their temporal order.

To test this method, use it in one final main class as shown in the following listing.

Listing 17.17. A main class printing the temperatures in three towns

```
public class Main {

    public static void main(String[] args) {
        Observable<TempInfo> observable = getCelsiusTemperatures(
            "New York", "Chicago", "San Francisco"
        );
        observable.blockingSubscribe( new TempObserver() );
    }
}
```

Other formats available



one in [listing 17.14](#) except that you're now subscribing to the `Observable` returned by the `getCelsiusTemperatures` method in [listing 17.16](#) and printing the temperatures registered for three towns. Running this main produces output such as this:

```
New York : 21
Chicago : 6
San Francisco : -15
New York : -3
Chicago : 12
San Francisco : 5
Got problem: Error!
```

Each second, `main` prints the temperature of each requested town until one of the temperature-fetching operations raises an error that's propagated to the `Observer`, interrupting the stream of data.

The purpose of this chapter wasn't to provide a complete overview of RxJava (or any other reactive library), for which a complete book would be necessary, but to give you a feeling for how this kind of toolkit works and to introduce you to the principles of reactive programming. We've merely scratched the surface of this programming style, but we hope that we've demonstrated some of its advantages and stimulated your curiosity about it.

Summary

- The fundamental ideas behind reactive programming are 20 to 30 years old, but have become popular recently because of the high demands of modern applications in terms of amount of processed data and users' expectations.
- These ideas have been formalized by the Reactive Manifesto, which states that reactive software must be characterized by four interrelated features: responsiveness, resiliency, elasticity, and that quality of being message-driven.
- The principles of reactive programming can be applied, with some differences, in implementing a single application and in designing a reactive system that integrates multiple applications.
- A reactive application is based on asynchronous processing of one or more flows of events conveyed by reactive streams. Because the role of reactive streams is so central in development of reactive applications, a consortium of companies including Netflix, Pivotal, Lightbend, and Red Hat standardized the concepts to maximize the interoperability of different implementations.
- Because reactive streams are processed asynchronously, they've been

Other formats available



- ✗ pressure mechanism. This prevents a overwhelmed by faster producers. standardization process has been incorporated into java. The java 9 Flow API defines four core interfaces: Publisher, Subscriber, Subscription, and Processor.
- These interfaces aren't intended, in most cases, to be implemented directly by developers, but to act as a lingua franca for the various li-

braries that implement the reactive paradigm.

- One of the most commonly used of these toolkits is RxJava, which (in addition to the basic features defined by the Java 9 Flow API) provides many useful, powerful operators. Examples include operators that conveniently transform and filter the elements published by a single reactive stream and operators that combine and aggregate multiple streams.

Other formats available



Audiobook

